

Guide & Master Roadmap: Architecture AIOps

Plateforme DevOps & Détection d'Anomalies dans les Logs

Version Finale Consolidée

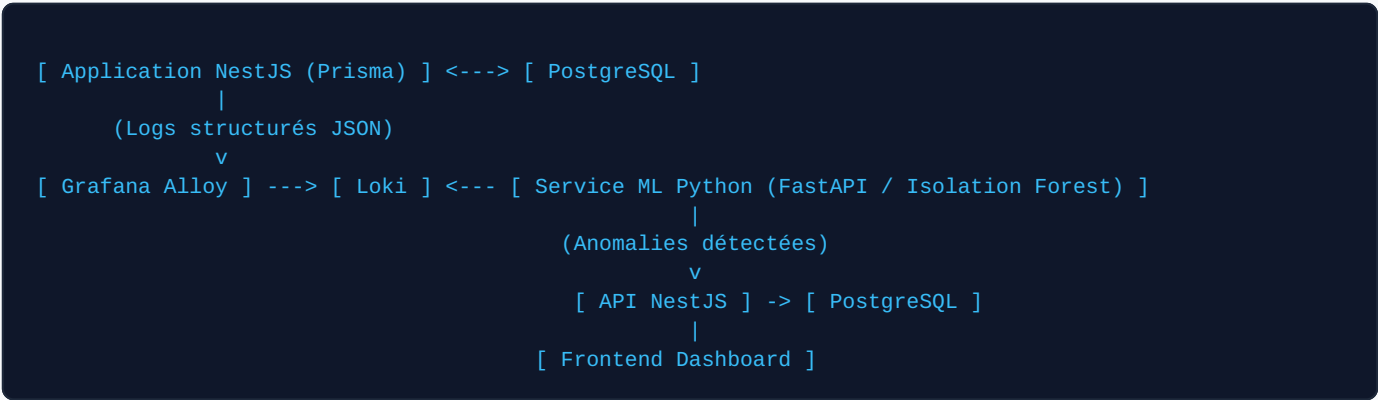
Septembre 2026 – Janvier 2027

AIOps / Observabilité

Principe Directeur du Projet : L'apprentissage prime sur la vitesse. La deadline est fixée à fin janvier 2027 (~20.5 semaines) pour permettre d'approfondir chaque brique technique (DevOps, IaC, ML, Observabilité) sur un stack self-hosted 100% gratuit.

1. Vision Globale & Stack Technique

Le système AIOps repose sur une architecture distribuée, conteneurisée et observable articulée autour de 4 blocs principaux (Backend/Database, Observabilité Logs & Métriques, Engine ML, Infrastructure/Orchestration) :



Composant	Technologie	Rôle Principal dans l'Architecture
Backend / API	NestJS (Node.js) + Prisma	API REST générant des logs JSON structurés et gestion du stockage des anomalies.
Base de Données	PostgreSQL	Stockage relationnel transactionnel des métadonnées applicatives et anomalies.
Log Ingestion	Grafana Alloy + Loki	Collecte (Alloy), stockage centralisé indexé par labels et requêtage LogQL (Loki).
Engine ML / AIOps	Python (FastAPI + Scikit-Learn)	Extraction de features et détection d'anomalies non supervisée via Isolation Forest.
Frontend	Dashboard (React / Angular)	Visualisation des anomalies (polling), graphique horaire, gestion des faux positifs.
Metrics & Alerting	Prometheus + Grafana + Alertmanager	Scraping des métriques app/ML, dashboards Grafana et webhooks Slack/Discord.
DevOps & IaC	Docker, Kubernetes (kind), Terraform, GitHub Actions	Conteneurisation multi-stage, orchestration local-first et pipeline CI/CD.

2. Fondations & Objectifs Stratégiques

A. Choix Techniques & Alignement Architecture

- Logs Structurés (JSON) :** Métadonnées explicites (timestamp, level, context) permettant un parsing optimal par Grafana Alloy et un stockage efficient dans Loki.

- **Isolation Forest (Non supervisé)** : Modèle ML idéal pour isoler des anomalies sans nécessiter de jeu de données pré-étiqueté.
- **Stack Self-Hosted / Local** : Fonctionnement intégralement gratuit sur cluster kind et Terraform, sans dépendance Cloud payante.

B. Profils Cibles & Attentes

Public Cible	Besoin & Valeur Ajoutée
Moi-même	Formation technique DevOps maximale (K8s, Terraform, Observabilité, ML).
Jury Doctorat ENSI Manouba	Projet démonstrateur avec dimension recherche (détection d'anomalies non supervisée).
Recruteurs DevOps	Portfolio complet démontrant une maîtrise DevOps réelle et éprouvée.
Enseignants-chercheurs (CRISTAL)	Démonstration concrète de compétences combinées en observabilité et Machine Learning.

3. Périmètre & Exigences Non-Fonctionnelles

Périmètre Inclus (In-Scope)	Périmètre Exclu (Out-of-Scope)
• API NestJS REST avec logs JSON • Base PostgreSQL via Prisma • Collecte Loki + Grafana Alloy • Service ML FastAPI (Isolation Forest) • Docker multi-stage & Cluster kind • Infrastructure as Code (Terraform) • Pipeline CI/CD GitHub Actions • Prometheus + Grafana + Alertmanager • Frontend dashboard minimal	• Deep Learning / Transformers / LLM • Déploiement multi-cloud complexe • Authentification OAuth / JWT lourde • Traitement de logs temps réel via Kafka • Offres Cloud managées payantes (Grafana Cloud, SaaS)

Exigences Non-Fonctionnelles Césures

- **Reproductibilité** : Déploiement 100% automatisé via terraform apply + kubectl apply.
- **Latence ML** : Détection et écriture en BDD en moins de 5 secondes après l'émission du log.
- **Précision ML** : Modèle atteignant au minimum 80% de détection sur dataset de test simulé.
- **Performance Frontend** : Chargement et affichage des données en moins de 2 secondes.
- **Alerting Critique** : Envoi d'une alerte Slack/Discord si le score d'anomalie dépasse 0.8.

4. Master Roadmap de Construction (Phases 1 à 12)

Plan d'exécution sur 20.5 semaines réparties en jalons progressifs avec Definition of Done (DoD) :

Phase 1 — Application NestJS + Prisma + PostgreSQL	1 Semaine
À construire : API REST NestJS basique, schéma Prisma (2-3 modèles), logger structuré JSON (nestjs-pino/winston), simulation d'erreurs et tests Jest. Connaissances : Format JSON structuré vs console.log, niveaux de logs (info, warn, error), migrations Prisma. DoD : API tourne en local, génère des logs JSON structurés contenant timestamp, level, message, context.	
Phase 2 — Conteneurisation locale (DevOps #1)	0.5 Semaine
À construire : Dockerfile multi-stage pour l'API NestJS, orchestre docker-compose.yml (API + DB + variables d'environnement).	

Connaissances : Multi-stage builds, networking Docker Compose, persistance via volumes PostgreSQL.

DoD : `docker compose up` démarre l'API et la BDD, endpoints accessibles de l'extérieur.

Phase 3 — CI/CD basique GitHub Actions (DevOps #2) 1.5 Semaine

À construire : Workflow GitHub Actions sur push (build Docker, tests Jest, push vers GitHub Container Registry).

Connaissances : Workflows YAML GitHub Actions, secrets CI/CD, mise en cache du cache npm.

DoD : Chaque push déclenche automatiquement build + tests + push d'image sans intervention.

Phase 4 — Kubernetes local (kind) + Terraform (DevOps #3) 2 Semaines

À construire : Cluster Kubernetes local via kind, manifests K8s (Pod, Deployment, Service), scripts Terraform provisionnant le cluster.

Connaissances : Concepts K8s, cycle Terraform (init, plan, apply, destroy), état `.tfstate`, ConfigMaps/Secrets.

DoD : `terraform apply` crée un cluster kind prêt, un Pod de test répond, `destroy` détruit tout proprement.

Phase 5 — Déploiement de l'application sur Kubernetes (DevOps #4) 2 Semaines

À construire : Deployment + Service NestJS, StatefulSet + PVC PostgreSQL, Ingress Nginx, health checks (liveness/readiness probes).

Connaissances : StatefulSet vs Deployment pour BDD, encodage des secrets, Rolling Updates K8s.

DoD : Application accessible via Ingress (`http://localhost`), données persistantes, CI/CD redéploie sur K8s.

Phase 6 — Observabilité des logs : Loki + Grafana Alloy 2.5 Semaines

À construire : Déploiement Helm de Loki et Grafana Alloy, parsing des logs JSON NestJS par Alloy, requêtes LogQL dans Grafana.

Connaissances : Architecture Loki vs Elasticsearch, syntaxe LogQL, pipeline Agent (Alloy) -> Stockage (Loki).

DoD : Logs de l'API visibles et filtrables dans Grafana via Loki, au moins 3 requêtes LogQL documentées.

Phase 7 — Service ML : Isolation Forest en local 2 Semaines

À construire : Script de génération de logs simulés, module de feature engineering, modèle Isolation Forest (scikit-learn), API FastAPI (endpoint `/predict`).

Connaissances : Feature engineering sur texte semi-structuré, algorithme Isolation Forest, détection non supervisée.

DoD : Service ML local répond sur `/predict` avec un score d'anomalie cohérent, précision > 80%.

Phase 8 — Intégration ML ↔ Loki ↔ PostgreSQL 2 Semaines

À construire : Polling du service ML vers l'API Loki, extraction/prédiction, envoi des anomalies à l'API NestJS/Prisma, déploiement K8s du service ML.

Connaissances : Communication DNS interne K8s, pattern polling vs event-driven, API HTTP Loki.

DoD : Un log anormal injecté est détecté par le service ML et stocké dans PostgreSQL en < 5 secondes.

Phase 9 — Monitoring métriques : Prometheus + Grafana 2 Semaines

À construire : Instrumentation Prometheus dans NestJS et FastAPI, déploiement Helm Prometheus, dashboards Grafana unifiés.

Connaissances : Métriques (Counter, Gauge, Histogram) vs Logs, langage de requête PromQL.

DoD : Dashboards Grafana affichent les métriques en temps réel avec au moins une requête PromQL complexe.

Phase 10 — Dashboard Frontend minimal 1.5 Semaine

À construire : Interface web SPA (React/Angular) listant les anomalies, polling périodique, graphique horaire, bouton "faux positif".

Connaissances : Consommation d'APIs REST, mise en place de rafraîchissement par polling.
DoD : Dashboard affiche les anomalies de la BDD, se rafraîchit automatiquement, interaction "faux positif" fonctionnelle.

Phase 11 — Alerting : Alertmanager + Webhooks 2 Semaines

À construire : Déploiement Alertmanager, règles d'alerte sur score d'anomalie (> 0.8), routing vers Webhook Slack/Discord.
Connaissances : Évaluation Prometheus vs Routing Alertmanager, principes de grouping/silencing d'alertes.
DoD : Une anomalie critique simulée déclenche une notification Slack/Discord en moins d'une minute.

Phase 12 — Documentation & Valorisation 1.5 Semaine

À construire : README complet (architecture, instructions terraform apply + kubectl apply), schémas affinés, nettoyage du repo.
Connaissances : Rédaction technique, vulgarisation d'architecture complexes.
DoD : Un tiers peut cloner le repository et exécuter l'intégralité de la plateforme en local sans assistance.

5. Questions de Recherche & Compétences Acquisées

Questions de Recherche (Doctorat ENSI)

- 1. Efficacité ML vs Règles Statiques** : Dans quelle mesure la détection non supervisée (Isolation Forest) améliore-t-elle la réactivité face aux incidents par rapport aux seuils fixes ?
- 2. Importance des Features** : Quelles caractéristiques extraites des logs (longueur, mots-clés, stacktrace, timestamp) sont les plus prédictives d'une anomalie ?
- 3. Apprentissage Continu / Feedback** : Comment l'intégration du feedback utilisateur ("faux positif") peut-elle affiner la précision du modèle dans le temps ?
- 4. Impact Performance** : Comment intégrer un pipeline ML au sein d'un cluster DevOps sans dégrader les performances globales (latence, mémoire, charge) ?

Synthèse des Compétences Acquisées

Domaine	Compétences Clés Maîtrisées
DevOps & IaC	Docker (multi-stage), Kubernetes (kind, Deployments, StatefulSets, Ingress), Terraform (IaC, state, modules), GitHub Actions.
Observabilité	Loki, Grafana Alloy, LogQL, Prometheus, PromQL, Grafana Dashboards, Alertmanager.
Backend & BDD	NestJS, Prisma ORM, PostgreSQL, API REST, logs JSON structurés.
ML / AIOps	Isolation Forest, Scikit-learn, Feature Engineering sur texte, FastAPI, Apprentissage non supervisé.